

JAVA SE 8+ REFERENCE : Date & Time API

Guide de référence complet, architecture, concepts et cas d'usage pratiques (`java.time`)

1. Introduction

Depuis Java 8, Java fournit une nouvelle API moderne pour manipuler les dates et les heures `java.time`. Elle remplace avantageusement les anciennes classes `java.util.Date` et `java.util.Calendar` qui souffraient de graves défauts de conception (mutabilité, thread-safety, manque de clarté).

Cette API permet de travailler de façon expresse et sécurisée avec :

- Les dates (sans heure)
- Les heures (sans date)
- Les dates + heures
- Les timestamps (instants précis)
- Les fuseaux horaires
- Les durées (secondes, nanosecondes)
- Les périodes (années, mois, jours)
- Le formatage et le parsing
- Les unités et champs temporels

Principe clé : Immutabilité et Thread-Safety

L'API est basée principalement sur le calendrier ISO-8601. Ses classes principales sont immutables et thread-safe, garantissant qu'aucune opération ne modifie l'instance existante mais retourne systématiquement une nouvelle instance.

2. Les principaux packages

L'API Date/Time est principalement organisée autour de trois packages principaux :

PACKAGE	RÔLE	CLASSES PRINCIPALES
<code>java.time</code>	Classes principales pour dates, heures, instants, fuseaux et durées.	<code>LocalDate</code> , <code>LocalTime</code> , <code>LocalDateTime</code> , <code>Instant</code> , <code>ZonedDateTime</code> , <code>Period</code> , <code>Duration</code>
<code>java.time.format</code>	Formatage et parsing des objets temporels.	<code>DateTimeFormatter</code> , <code>DateTimeFormatterBuilder</code>
<code>java.time.temporal</code>	Champs, unités, ajustements avancés et abstractions temporelles.	<code>ChronoUnit</code> , <code>ChronoField</code> , <code>TemporalAdjusters</code> , <code>TemporalQuery</code>

3. java.time — Les types de base

3.1 LocalDate

`LocalDate` représente une date pure (année, mois, jour) sans heure ni fuseau horaire. Recommandé par Oracle pour des concepts calendaires tels qu'une date de naissance ou une date de réservation.

```
import java.time.LocalDate;

// Création d'une date spécifique
LocalDate date = LocalDate.of(2026, 9, 7);
System.out.println(date); // 2026-09-07

// Date actuelle
LocalDate today = LocalDate.now();

// Opérations d'addition et de soustraction
LocalDate newDate = date.plusDays(10); // 2026-09-17
LocalDate previous = date.minusDays(5); // 2026-09-02

// Autres opérations courantes
date.plusWeeks(2);
date.minusWeeks(2);
date.plusMonths(3);
date.minusMonths(3);
date.plusYears(1);
date.minusYears(1);
```

3.2 LocalTime

`LocalTime` représente une heure sans date ni fuseau horaire (ex. heures d'ouverture d'un magasin).

```
import java.time.LocalDateTime;

LocalTime time = LocalDateTime.of(14, 30);
System.out.println(time); // 14:30

LocalTime now = LocalDateTime.now();

// Manipulation du temps
time.plusHours(2).plusMinutes(15).plusSeconds(30);

// Exemple métier : Horaires de magasin
LocalTime openingTime = LocalDateTime.of(9, 0);
LocalTime closingTime = LocalDateTime.of(18, 0);
```

3.3 LocalDateTime

`LocalDateTime` combine une date et une heure (`LocalDate` + `LocalTime`), mais toujours sans fuseau horaire.

```
import java.time.LocalDateTime;

LocalDateTime dateTime = LocalDateTime.of(2026, 9, 7, 14, 30);
System.out.println(dateTime); // 2026-09-07T14:30

LocalDateTime now = LocalDateTime.now();

// Extraction d'informations
int year = dateTime.getYear();
int month = dateTime.getMonthValue();
int day = dateTime.getDayOfMonth();
int hour = dateTime.getHour();
int minute = dateTime.getMinute();
```

4. Gestion de la Timeline et Fuseaux Horaires

4.1 Instant

`Instant` représente un point précis et absolu sur la ligne du temps (timestamp UNIX depuis le 01/01/1970 00:00:00 UTC). Idéal pour les logs, les métriques techniques, et la persistance en base de données.

```
import java.time.Instant;

Instant now = Instant.now();
System.out.println(now); // Ex: 2026-09-07T13:30:45.123Z ('Z' = UTC)
```

Comparaison essentielle :

- `LocalDateTime` : Date + heure locale globale (ex: un rendez-vous le 07/09 à 14:30 quel que soit le pays).
- `Instant` : Moment physique universel sur la timeline UTC.

4.2 ZoneId, ZonedDateTime et OffsetDateTime

Pour lier une heure locale à un endroit précis du globe, Java fournit les abstractions de fuseaux :

- **ZoneId** : Identifiant de zone géographique (ex: `"Africa/Casablanca"` , `"Europe/Paris"`). Gère les règles complexes de changement d'heure (DST).
- **ZonedDateTime** : Date + Heure + Fuseau complet avec règles (`ZoneId`).
- **OffsetDateTime** : Date + Heure + Décalage fixe par rapport à UTC (ex: `+01:00`).

```
import java.time.*;

// Utilisation de ZoneId
ZoneId zoneCasablanca = ZoneId.of("Africa/Casablanca");

// Date et heure zonées
ZonedDateTime zdtCasablanca = ZonedDateTime.now(zoneCasablanca);

// Conversion vers un autre fuseau (même instant temporel)
ZonedDateTime zdtParis = zdtCasablanca.withZoneSameInstant(ZoneId.of("Europe/Paris"));

// OffsetDateTime
OffsetDateTime odt = OffsetDateTime.now(); // Ex: 2026-09-07T14:30:00+01:00
```

5. Quantités de Temps : Period vs Duration

Java distingue les quantités de temps calendaires des quantités de temps physiques :

CONCEPT	PERIOD	DURATION
Unités	Années, Mois, Jours	Secondes, Nanosecondes, Minutes, Heures
Domaine	Temps Calendaire (basé sur le calendrier humain)	Timeline Physique (basé sur l'horloge système)
Exemple	<code>Period.ofMonths(3)</code> (Âge, préavis)	<code>Duration.ofHours(5)</code> (Temps d'exécution, latence)

```
import java.time.*;

// Period : Calcul de l'âge ou différence en jours/mois/années
LocalDate birthDate = LocalDate.of(1990, 5, 10);
LocalDate today = LocalDate.now();
Period age = Period.between(birthDate, today);
System.out.println("Âge : " + age.getYears() + " ans");

// Duration : Calcul de durée entre deux temps
LocalTime start = LocalTime.of(10, 0);
LocalTime end = LocalTime.of(12, 30);
Duration duration = Duration.between(start, end);
System.out.println(duration); // PT2H30M
System.out.println("Minutes totales : " + duration.toMinutes()); // 150
```

6. java.time.format — Formatage et Parsing

La classe `DateTimeFormatter` permet la conversion bidirectionnelle entre objets temporels et chaînes de caractères.

Symboles de formatage :

- `dd` : Jour du mois (01-31)
- `MM` : Mois (01-12)
- `yyyy` : Année à 4 chiffres
- `HH` : Heure (00-23)
- `mm` : Minutes (00-59)
- `ss` : Secondes (00-59)

DateTimeFormatterBuilder : Quand un pattern standard ne suffit pas, `DateTimeFormatterBuilder` permet de construire des formatters sur-mesure complexes (gestion des majuscules/minuscules, valeurs par défaut, etc.).

```
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

// 1. Formatage (Objet -> String)
LocalDate date = LocalDate.of(2026, 9, 7);
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String formattedDate = date.format(formatter); // "07/09/2026"

// 2. Parsing personnalisé (String -> Objet)
String text = "07/09/2026";
LocalDate parsedDate = LocalDate.parse(text, formatter); // 2026-09-07

// 3. Parsing ISO automatique
LocalDate isoDate = LocalDate.parse("2026-09-07");
LocalTime isoTime = LocalTime.parse("14:30:00");
LocalDateTime isoDateTime = LocalDateTime.parse("2026-09-07T14:30:00");
```

7. java.time.temporal — Traitements Avancés

7.1 ChronoUnit vs ChronoField

TYPE	RÔLE / QUESTION	EXEMPLES
<code>ChronoUnit</code>	UNITÉ — Dans quelle unité je calcule ?	<code>DAYS</code> , <code>MONTHS</code> , <code>YEARS</code> , <code>HOURS</code>
<code>ChronoField</code>	CHAMP — Quel champ de la date je lis/modifie ?	<code>YEAR</code> , <code>MONTH_OF_YEAR</code> , <code>DAY_OF_WEEK</code>

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.time.temporal.ChronoField;

LocalDate start = LocalDate.of(2026, 9, 1);
LocalDate end = LocalDate.of(2026, 9, 7);

// Calcul d'écart via ChronoUnit
long daysBetween = ChronoUnit.DAYS.between(start, end); // 6

// Lecture de champ via ChronoField
int year = end.get(ChronoField.YEAR); // 2026
int month = end.get(ChronoField.MONTH_OF_YEAR); // 9
```

7.2 TemporalAdjusters & Abstractions

La classe `TemporalAdjusters` fournit des stratégies d'ajustement temporel courantes :

```
import java.time.LocalDate;
import java.time.DayOfWeek;
import java.time.temporal.TemporalAdjusters;

LocalDate date = LocalDate.of(2026, 9, 7);

// Ajustements automatiques
LocalDate lastDay = date.with(TemporalAdjusters.lastDayOfMonth()); // Dernier jour du mois
LocalDate firstDay = date.with(TemporalAdjusters.firstDayOfMonth()); // Premier jour du mois
LocalDate nextMonday = date.with(TemporalAdjusters.next(DayOfWeek.MONDAY)); // Prochain lundi
```

Abstractions clés du package temporel :

- `Temporal` : Interface représentant un objet temporel modifiable (via `with()` , `plus()`).
- `TemporalAccessor` : Interface d'accès en lecture seule aux champs temporels.
- `TemporalAdjuster` : Interface pour personnaliser des stratégies d'ajustement.
- `TemporalQuery` : Permet de requêter des informations complexes sur un objet temporel.

8. Nomenclatures et Conventions des Méthodes

Oracle applique une convention de nommage très rigoureuse et uniforme à travers toute l'API :

PRÉFIXE	SIGNIFICATION	EXEMPLE
<code>of()</code>	Fabrique statique de création	<code>LocalDate.of(2026, 9, 7)</code>
<code>now()</code>	Obtenir l'instant courant	<code>LocalDateTime.now()</code>
<code>parse()</code>	Convertir une chaîne en objet	<code>LocalDate.parse("2026-09-07")</code>
<code>get()</code>	Lire la valeur d'un champ	<code>date.get(ChronoField.YEAR)</code>
<code>is...()</code>	Vérification / Prédicat	<code>date.isBefore(other)</code> , <code>date.isAfter(other)</code>
<code>with()</code>	Créer une copie modifiée d'un champ	<code>date.withYear(2030)</code>
<code>plus()</code> / <code>minus()</code>	Ajouter ou soustraire du temps	<code>date.plusDays(10)</code> , <code>date.minusMonths(2)</code>
<code>to...()</code>	Convertir vers un autre type	<code>dateTime.toLocalDate()</code>
<code>at...()</code>	Combiner avec une autre entité	<code>date.atTime(14, 30)</code>

9. Guide de Sélection Rapide

Synthèse décisionnelle pour choisir le bon type selon le besoin fonctionnel :

BESOIN / CAS D'USAGE	TYPE / CLASSE À PRIVILÉGIER
Date de naissance, date de réservation	<code>LocalDate</code>
Heure d'ouverture de magasin	<code>LocalTime</code>
Date + heure locale sans fuseau (rendez-vous)	<code>LocalDateTime</code>
Timestamp technique, log, événement système	<code>Instant</code>
Événement multi-fuseaux avec règles DST	<code>ZonedDateTime</code>
Échange d'API REST avec décalage horaire	<code>OffsetDateTime</code>
Calcul d'âge ou période en années/mois/jours	<code>Period</code>
Durée d'exécution, chronométrage (secondes/ms)	<code>Duration</code>
Formatage et affichage utilisateur	<code>DateTimeFormatter</code>
Écart exact d'unités (ex: nombre de jours total)	<code>ChronoUnit</code>
Prochain lundi, dernier jour du mois	<code>TemporalAdjusters</code>

10. ARCHITECTURE MENTALE ET TOP 10 DES PRIORITÉS

10.1 Schéma d'Architecture Mentale

JAVA DATE & TIME API

```
├── java.time
│   ├── LocalDate
│   ├── LocalTime
│   ├── LocalDateTime
│   ├── Instant
│   ├── ZonedDateTime
│   ├── OffsetDateTime
│   ├── Period
│   └── Duration
├── java.time.format
│   └── DateTimeFormatter
└── java.time.temporal
    ├── ChronoUnit
    ├── ChronoField
    └── TemporalAdjusters
```

10.2 Les 10 éléments à maîtriser en priorité

- | | |
|---|--|
| 1. <code>LocalDate</code> | 1. <code>Period</code> |
| 2. <code>LocalTime</code> | 2. <code>Duration</code> |
| 3. <code>LocalDateTime</code> | 3. <code>Instant</code> |
| 4. <code>DateTimeFormatter</code> | 4. <code>ZonedDateTime</code> / <code>ZoneId</code> |
| 5. <code>Parsing</code> / <code>Formatting</code> | 5. <code>ChronoUnit</code> / <code>ChronoField</code> / <code>TemporalAdjusters</code> |

11. Exemple Complet de Synthèse

```
import java.time.LocalDate;
import java.time.Period;
import java.time.format.DateTimeFormatter;

public class Main {
    public static void main(String[] args) {
        // Date de naissance
        LocalDate birthDate = LocalDate.of(1990, 5, 10);

        // Date actuelle
        LocalDate today = LocalDate.now();

        // Calcul de l'âge
        Period age = Period.between(birthDate, today);
        System.out.println("Âge : " + age.getYears() + " ans");

        // Formatage
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        System.out.println("Naissance : " + birthDate.format(formatter));
    }
}
```